Enigma Engine

Project Presentation and User Manual

Interactive Graphics – Final Project

Contents

1 Introduction

Enigma Engine is a browser-based, real-time 3D car experience built with WebGL. It lets the user drive a car around a race track using a physics-based driving model, customize the car's paint and materials, open its doors and windows, control its lights, switch between several camera views (orbit, free camera, driver view, drone view), while tracking your best lap time. A dashboard sidebar exposes all of these options through a simple graphical interface, and a heads-up display shows live telemetry (speed, RPM, gear, lap time).

Everything runs directly in the browser: there is no installation step, no backend server, and no build/compile step. This document is both a technical presentation of how the project is built and a short user manual explaining how to use it.

2 Environment

2.1 Runtime and module loading

The project is built on top of **WebGL**, accessed through the **Three.js** rendering library (version 0.160.0). The application is a single HTML page (`index.html`) that loads plain JavaScript ES modules directly, with no bundler (Webpack, Vite, Parcel, ...) and no package manager (no `node_modules`, no `npm install`). This keeps the project extremely easy to run: opening `index.html` in a local static server is enough.

All third-party libraries are declared once, in the page's *import map*, and are downloaded from public content delivery networks (unpkg and jsDelivr) the first time the page is opened:

```
"three":            "https://unpkg.com/three@0.160.0/build/three.module.js"
"three/addons/":    "https://unpkg.com/three@0.160.0/examples/jsm/"
"@tweenjs/tween.js":"https://unpkg.com/@tweenjs/tween.js@23.1.1/..."
"cannon-es":        "https://cdn.jsdelivr.net/npm/cannon-es@0.20.0/+esm"
```

Every JavaScript file in the project then simply writes `import * as THREE from 'three'` (or similar), exactly as it would with a bundler, but without any local installation step.

2.2 Rendering pipeline configuration

The renderer is configured for a realistic look:

- Antialiasing enabled, `powerPreference: "high-performance"` to request the discrete GPU when available.
- Output color space set to sRGB, with ACES filmic tone mapping (exposure 1.0) for a more cinematic contrast curve.
- Shadows enabled with a soft PCF shadow map.

The camera is a standard perspective camera with a 60° field of view and near/far clipping planes at 0.1 and 1000 units.

2.3 Application startup sequence

When the page loads, the application: (1) creates the scene, camera and renderer; (2) loads the track and the car model in parallel; (3) builds the physics world and the car's rigid body from the loaded meshes; (4) wires up every UI control (lights, colors, animations, camera buttons, HUD);

(5) runs a short *prewarm* pass that forces all shaders to compile and renders a few warm-up frames (both in day and night lighting) so that the GPU pipeline is fully "hot" before anything is shown; and finally (6) hides the loading overlay and starts the real render loop. This prewarm step avoids a visible stutter on the very first frames, which is otherwise a common issue with WebGL applications that compile shaders lazily.

3 Libraries, Tools and Models Not Developed by the Team

The following third-party software and assets are used in the project but were not created by the team:

- **Three.js** (v0.160.0) – WebGL rendering engine, together with its official add-ons: `GLTFLoader` and `DRACOLoader` (to load the compressed 3D models) and `OrbitControls` (to orbit the camera around the car).
- **Draco** – Google's mesh compression decoder (hosted on Google's own CDN), used by `GLTFLoader` to decompress the 3D model geometry when it is loaded.
- **tween.js** (v23.1.1) – animation/tweening library used to smoothly animate doors, windows, lights intensity and camera transitions between two states.
- **cannon-es** (v0.20.0) – physics engine used for the whole vehicle simulation: rigid bodies, gravity, suspension, wheel raycasting and collisions with the track.
- **Car model** (`car.glb`, ~18 MB) – a detailed car model obtained from an external source, not modeled by the team; its exported material file still references the original Blender project and artist folder names.
- **Track model** (`track_1.glb`, ~69 MB) – the race circuit and its surrounding environment (road, trees, lamp posts, garage), also an external asset.
- **Textures** – the day/night equirectangular sky panoramas and the PBR ground/floor textures (diffuse, normal and roughness maps) are free texture packs, not created by the team.
- **Audio samples** – the engine sound recordings (five samples captured at fixed RPM values, from idle to redline) and the short sound effects (door open/close, turn signal click, engine startup) are external audio files, not recorded by the team.

Everything else – the whole application logic, the physics setup, the lighting and reflection system, the UI, the audio synthesis, and the way all of the above assets are combined – was written by the team.

4 Project Structure

The repository is organized into two top-level folders:

```
final-project-enigma/
|-- code/
|   |-- page/
|   |   |-- index.html      # entry point, import map, UI markup
|   |   \-- style.css       # UI/HUD styling
|   \-- script/             # ES modules, no bundler
```

```
|       |-- animations.js   # part animation + click-to-interact system
|       |-- audio.js        # Web Audio SFX and engine sound synthesis
|       |-- bestLap.js      # lap timing via a trigger zone
|       |-- camera.js       # orbit / free / driver / drone camera modes
|       |-- car_model.js    # declarative description of animatable parts
|       |-- color.js        # material registry, live color/roughness edit
|       |-- engine.js       # engine and gearbox simulation
|       |-- environment.js  # track loading, sky textures, tree instancing
|       |-- lights.js       # scene lights, car lights, street lamp pool
|       |-- loaders.js      # shared GLTFLoader + DRACOLoader
|       |-- main.js         # bootstraps the app, render/animate loop
|       |-- model.js        # generic GLTF loader + per-part tweening
|       |-- physics.js      # vehicle physics (RaycastVehicle)
|       |-- reflections.js  # dynamic cube-map reflections
|       |-- scene.js        # scene, camera and renderer setup
|       |-- settings.js     # small global settings store
|       \-- ui.js           # DOM event wiring for sidebar and HUD
\-- src/
    |-- models/             # car and track GLB files
    |-- textures/           # sky and PBR ground/floor textures
    \-- audio/               # engine RPM samples and short SFX
```

Each JavaScript module has a single responsibility, and `main.js` only wires the modules together: it does not itself contain rendering, physics or UI logic.

5 Technical Aspects

5.1 Scene, camera and the render loop

The scene uses a perspective camera and the WebGL renderer described in Section 2.2. The main render loop runs on `requestAnimationFrame`. On every frame it, in order:

1. advances the physics simulation by one step;
2. updates the on-screen speed / RPM / gear readouts;
3. updates the procedural engine sound based on the new RPM;
4. moves the sun light so it stays above and around the car;
5. updates the active camera mode (orbit, free, driver or drone);
6. advances any running tween animation (doors, lights, transitions);
7. refreshes the car's dynamic reflections and the nearby street lamps;
8. checks the lap-timer trigger zone against the car's position;
9. renders the frame.

A small FPS counter (updated once per second) is shown in the corner for debugging and performance monitoring.

5.2 Vehicle physics

The car is simulated as a rigid body using `cannon-es`'s `RaycastVehicle`: a box-shaped chassis with four independently suspended wheels. Table 1 summarizes the main tuning parameters.

Table 1: Wheel and suspension parameters

Parameter	Front wheels	Rear wheels
Radius (m)	0.338	0.345
Suspension rest length (m)	0.22	
Suspension stiffness	35	38
Damping (compression / relaxation)	2.5 / 3.8	2.3 / 3.6
Friction slip	4.5	4.5
Max steering angle	25°	–

The chassis is a box of roughly $0.90 \times 0.60 \times 2.25$ metres with a mass of 1500 kg, spawned at a fixed point on the track. Gravity is set to $-9.82\,\mathrm{m/s^2}$ and the physics world steps at a fixed 60 Hz with sub-stepping, independently of the rendering frame rate.

The track geometry is converted into a static physical collision mesh: at load time, every track mesh that is not a tree is turned into an exact `Trimesh` collider, so the car collides with the real shape of the road, curbs and walls instead of a simplified approximation.

Several extra behaviors sit on top of the base physics simulation:

- **Smoothed steering**: the steering angle does not snap instantly to the key input, it eases toward its target and back to center, capped at ±25°.
- **Speed-dependent understeer**: above roughly 50 km/h, if the requested steering angle is large relative to the current speed, the front wheels' grip and effective steering angle are both reduced, so the car cannot corner unrealistically sharply at high speed.
- **Low-speed brake lock**: when the car is nearly stopped and the brake is held with no throttle, its remaining velocity is zeroed directly, so it does not creep forever at a tiny residual speed.
- **Rollover recovery**: if the car's "up" direction tips more than about 101° from vertical (i.e. it has flipped), it is automatically lifted, its roll and pitch are reset to level (yaw is kept), and its velocity is cleared, so the player is never permanently stuck upside down.

5.3 Engine and transmission model

On top of the physics engine, a hand-written engine model computes the wheel force from a torque curve and a gearbox, instead of relying on a single generic acceleration value. The torque curve is a lookup table, linearly interpolated between the sampled points shown in Table 2.

Table 2: Torque curve (RPM → Nm)

RPM	800	1500	2000	3000	4000	5000
Nm	220	260	290	350	400	440
RPM	6000	7000	8000	8500	9000	9200
Nm	470	460	430	390	340	150

The gearbox has six forward gears plus reverse, with the ratios in Table 3 (all multiplied by a fixed final-drive ratio of 3.96):

Table 3: Gear ratios

Gear	1	2	3	4	5	6
Ratio	4.75	3.08	2.40	2.04	1.68	1.18

The gearbox is fully automatic: it shifts up when the engine goes above about 7000 RPM, and shifts down when RPM drops below a threshold that also depends on how much throttle is applied (so a light touch of the throttle does not trigger an unwanted downshift). A short cooldown between shifts prevents rapid gear "hunting". Idle RPM is 800 and redline is 9000; engine RPM itself is smoothed with different response speeds when rising versus falling, to mimic engine inertia. The resulting wheel force is:

$$F_{\text{wheel}} = \frac{\text{torque}(RPM) \times \text{gear ratio} \times \text{final drive} \times \text{efficiency}}{\text{wheel radius}}$$

which is applied directly to the rear wheels of the `RaycastVehicle`.

5.4 Lighting system

The scene has a directional "sun" light (intensity 8.5, casting a 2048×2048 shadow map) which is re-targeted every frame to stay above the car, keeping shadow quality high near the player regardless of the track size. An ambient hemisphere light (intensity 1.2) provides soft indirect sky/ground lighting.

A day/night cycle smoothly interpolates a 0–24h virtual clock over about two seconds when the "Night Mode" switch is toggled. This clock drives: the sky texture (day or night equirectangular panorama), the sun and hemisphere light intensities (eased with a smoothstep curve), and whether the street lamps are active.

Each of the car's own lights (low/high beams, running lights, tail lights, brake lights, turn signals, interior ambient light) is built by locating the corresponding named mesh inside the 3D model, upgrading its material so it can glow (`emissive` property), and attaching a matching spot light. Turning a light on or off is animated with a short tween instead of an instant on/off, for a more natural look. Turn signals additionally blink on a fixed interval and play a click sound on every "on" phase.

Street lamps deserve a special mention: the track model can contain dozens of lamp posts, but creating one real light per lamp post would be too expensive. Instead, a fixed pool of 21 spotlights is created once; every frame, the lamp posts are sorted by distance to the car and the pool is re-assigned to the 21 closest ones. This keeps the lighting cost constant regardless of how large the track is.

5.5 Reflections

The car's paint reacts to its surroundings using a dynamic cube-map reflection: a `CubeCamera` renders the environment from a point just above the car into a small (64px) cube render target, which is then used as the environment map for the car's materials. Rendering a full cube map every frame would be expensive, so it is refreshed only once every three frames by default, and a "Static Reflections" setting lets the user freeze it completely for better performance on slower machines.

5.6 Model, materials and part-animation system

The car's animatable parts (doors, windows, hood, rear wing, ignition key, wheels, steering wheel) are described declaratively in a single configuration object, rather than being hard-coded one by one: each part lists which mesh it corresponds to, how it should rotate or move, how long the animation should take, which sidebar switch controls it, and which sound effects to play. A generic loader reads this configuration once the 3D model is loaded and precomputes the "closed" and "open" transform for every part, so that later toggling only needs to interpolate between two already-known states.

Materials are collected into a name-based registry as the car model is loaded (using the material names authored in Blender), which is what allows the sidebar's color pickers and metallic/roughness sliders to edit the live materials directly.

Clicking on a part in the 3D scene (instead of using the sidebar) works through raycasting: only the meshes that belong to a clickable part are considered, both for detecting hover (to draw a highlight) and for detecting clicks, which keeps this check fast even though the whole car model has many more meshes than just the clickable ones.

5.7 Audio system

All audio uses the native Web Audio API directly, with no external audio library. Short one-shot effects (door open/close, turn signal click, engine startup) are simply decoded once and played back on demand.

The running engine sound is synthesized rather than looped from a single clip: five samples were recorded at fixed RPM values (900, 2294, 4139, 6025, 8198), and at runtime the two samples that bracket the current RPM are cross-faded using an equal-power (cosine-based) curve, while each sample's playback rate is also adjusted so its pitch matches the exact current RPM. A second pair of gain buses blends between an "on-throttle" and an "off-throttle / coasting" mix depending on how much the accelerator is pressed, so the engine also sounds different when accelerating versus coasting at the same RPM.

5.8 Lap timing

Lap timing uses an invisible trigger volume placed at a "finish line" marker inside the track model. Every frame, the car's world position is converted into the trigger's local space and compared against its bounds; crossing into the zone for the first time starts the lap clock, and crossing it again on a later lap stops the current lap and compares it against the stored best time, updating the on-screen best lap (`mm:ss:ms`) if the new lap was faster.

5.9 Performance optimizations – summary

Several techniques throughout the project exist purely to keep the frame rate high:

- **GPU instancing** for trees: every tree of the same type on the track is drawn with a single `InstancedMesh` draw call instead of one draw call per tree, with small per-instance random rotation/scale for visual variety.
- **Light pooling** for street lamps (Section 5.4): a fixed number of real lights is reused for however many lamp posts exist.
- **Throttled and freezable reflections** (Section 5.5): the cube-map reflection is updated every few frames, or not at all in static mode.

- **Filtered raycasting** (Section 5.6): click/hover detection only tests the small set of clickable meshes, not the whole car model.
- **Shader/pipeline prewarming** (Section 2.3): shader compilation and the first heavy frames happen behind the loading screen, not during actual gameplay.

6 User Manual – Implemented Interactions

6.1 Getting started

Open `code/page/index.html` through a local web server (opening the file directly may be blocked by the browser's security policy for module imports). Wait for the loading screen to disappear, then press the **ENGINE** button in the instrument cluster to start the car, or press **E**, select **D** in the gear selector, and use WASD drive.

6.2 Driving controls

Table 4: Driving controls

Input	Action
W	Accelerate
S / Space	Brake
A / Right arrow	Steer
S / Space	Brake lights on (while held)
E / HUD engine button	Start / stop the engine (with crank sound)
HUD gear selector (N / D / R)	Neutral / Drive / Reverse
Left Shift	Upshift gears in order Reverse, Neutral, Drive
Left Control	Downshifts gears in order Drive, Neutral, Reverse

The engine only actually starts once the crank sound has finished playing, so there is a short delay between pressing the button and being able to move.

6.3 Camera controls

Table 5: Camera controls

Input / button	Action
Mouse drag (orbit mode)	Rotate the camera around the car
Mouse scroll (orbit mode)	Zoom in / out (clamped range)
Up/Left/Down/Right Arrow keys, Right/Left Control (free mode)	Fly forward/back/left/right, down/up
Mouse drag (free / driver mode)	Look around
"Orbit Camera" / "Free Camera" button	Toggle between orbit and free-fly
Compass buttons (Front/Back/Left/Right/Top)	Jump to a preset view
"Driver View" button	First-person onboard camera
"Drone View" button	Top-down camera following the car

Switching to a preset compass view plays a smooth, eased transition instead of an instant camera cut. Driver view lets the player look around with the mouse; releasing the mouse smoothly re-centers the view.

6.4 Car customization panel

The sidebar "Dashboard" is organized into collapsible sections:

- **Color Customization**: a color picker plus metallic and roughness sliders for the body paint, rims, brake calipers, seat fabric and steering wheel, applied live to the 3D model.
- **Animations**: switches to open/close the hood, rear wing, and both doors and windows. The same parts can also be opened by clicking on them directly in the 3D scene; hovering over a clickable part highlights it in blue.
- **Lights**: switches for running lights, low beams, high beams, left/right turn indicators, hazard lights and the interior ambient light. These switches follow realistic real-car dependencies: turning on the high beams automatically turns on the low beams, and turning off the running lights also turns off both beam types.
- **Settings**: a switch to enable/disable the blue hover highlight on clickable parts, and a "Static Reflections" switch that freezes the car's cube-map reflections to save performance.
- **Time of Day**: a single "Night Mode" switch that smoothly transitions the whole scene between day and night lighting over about two seconds.

6.5 Heads-up display

The main view overlays a small instrument cluster and telemetry panel:

- **Speed** in km/h.
- **RPM**, shown both as a number and as a color-coded rev bar (green → orange → red, flashing near the redline).
- **Gear** readout (current numeric gear in Drive, or N / R).
- **Best lap** time, in `mm:ss:ms` format, updated automatically whenever a faster lap is completed.
- **FPS counter**, updated once per second.

7 Conclusion

Enigma Engine combines a real-time WebGL renderer, a raycast-based vehicle physics simulation with a custom engine/gearbox model, a dynamic day/night lighting system, procedurally synthesized engine audio, and a fully interactive customization UI, all running client-side in the browser with no build step. The result is an interactive car showroom and driving simulator that can be opened and used directly from a static web page.